



PyHelp_csv_manager for HydroModPy

User guide and technical documentation

Version 0.1

1) General presentation	2
2) File organization and installation	2
3) Input data details	3
3.1 CSV input grid (for PyhelpGrid)	3
3.2 DEM (Digital Elevation Model)	3
3.3 Shapefile (study area)	3
3.4 ERA5 (netCDF files)	3
4) How the main script works	4
4.1 Loading and updating the grid	4
4.2 ERA5 data extraction and aggregation	4
4.3 Final CSV file format	5
5) Description of modules	6
5.1 helper.py	6
5.2 pyhelp_csv_manager.py	6
5.3 pyhelp_grid.py	6
5.4 pyhelp_era5.py	6
5.5 main.py	7
6) Outputs	7
6.1 Generated CSV Files (ERA5 Outputs)	7
6.2 Grid CSV File Updates	7

1) General presentation

The aim of this code is to prepare the input files for PyHelp, an object oriented Python library providing a set of tools to estimate spatially distributed groundwater recharge and other hydrological components. It revolves around two main python classes:

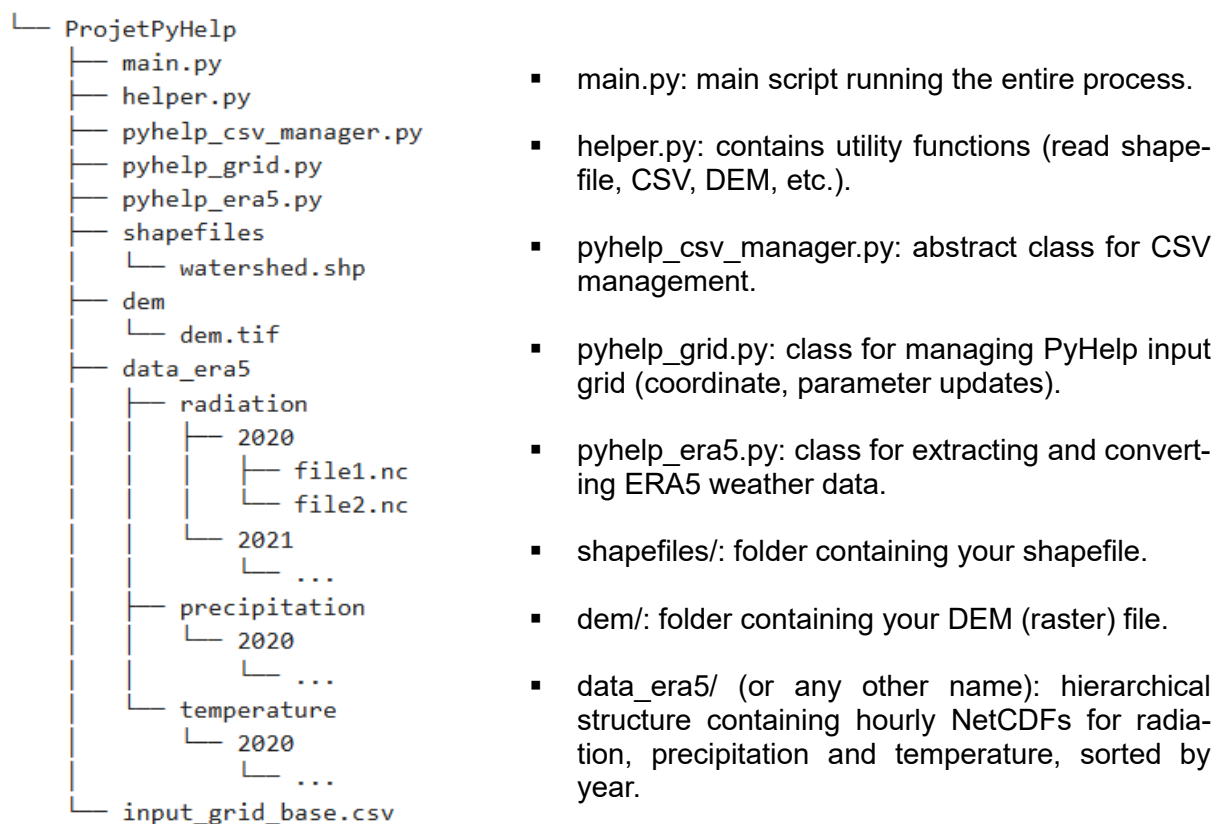
- 1) **PyhelpGrid**: Manages the PyHelp input grid CSV file, its parameters (wind, humidity, etc.) and grid coordinate update through a given DEM. This file is used by PyHelp to enter the grid and geomatics data.
- 2) **PyhelpEra5**: Reads and processes the given ERA5 NetCDF files (in hourly weather data), selects the point or zone corresponding to the polygon in a given shapefile, aggregates the data into daily averages, converts the units and exports the whole datasets in CSVs formatted for PyHelp (solrad_input_data.csv, precip_input_data.csv, air-temp_input_data.csv, etc.).

Both of those classes combined allow for an automated reorganization and aggregation of hourly data (NetCDF) into daily averages, unit conversion (temperature, precipitation, radiation) as well as an automatic update of the CSV grid by adding latitude and longitude coordinates from a DEM file given by the user.

2) File organization and installation

Working directory structure

The general structure is as follows:



Dependency installation

To run the code, you need a recent version of python and the following python libraries:

```
pip install pandas geopandas rasterio xarray pyproj
```

or with Conda :

```
conda install -c conda-forge pandas geopandas rasterio xarray pyproj
```

3) Input data details

3.1 CSV input grid (for PyhelpGrid)

Use the base CSV file (input_grid_base1.csv) specifically preformatted for the use of this class. This preformatted CSV file represents an homogeneous single layer case. It contains:

- Parametric columns (wind, hum1, growth_start, etc.).
- Latitude and longitude fields (which will be updated automatically).
- A "cid" identifier with a unique index.

When the script is run, this CSV is loaded and updated by the PyhelpGrid class by performing the following actions through its methods:

- `update_parameters`: assigns new values to the above-mentioned parameters.
- `transform_coordinates` : Reads the DEM and assigns coordinates (latitude, longitude) to each line, then rewrites the CSV.

3.2 DEM (Digital Elevation Model)

This raster file must be georeferenced in a certain EPSG for example 2056, the CH1903+ / LV95 system in Switzerland. The code will iterate through each of its pixels (row, column), extract the x, y coordinate, and then reproject it in EPSG:4326 (WGS 84). As a result, each pixel will then correspond to a row in the grid CSV.

3.3 Shapefile (study area)

This shapefile must contain the study area. The code will load the geometry (polygon) contained in the shapefile, Calculate its centroid (x, y) and select the nearest ERA5 point to filter and extract the netCDF data.

Work in progress : if the polygon is larger than, it will select the ERA5 points within its perimeter.

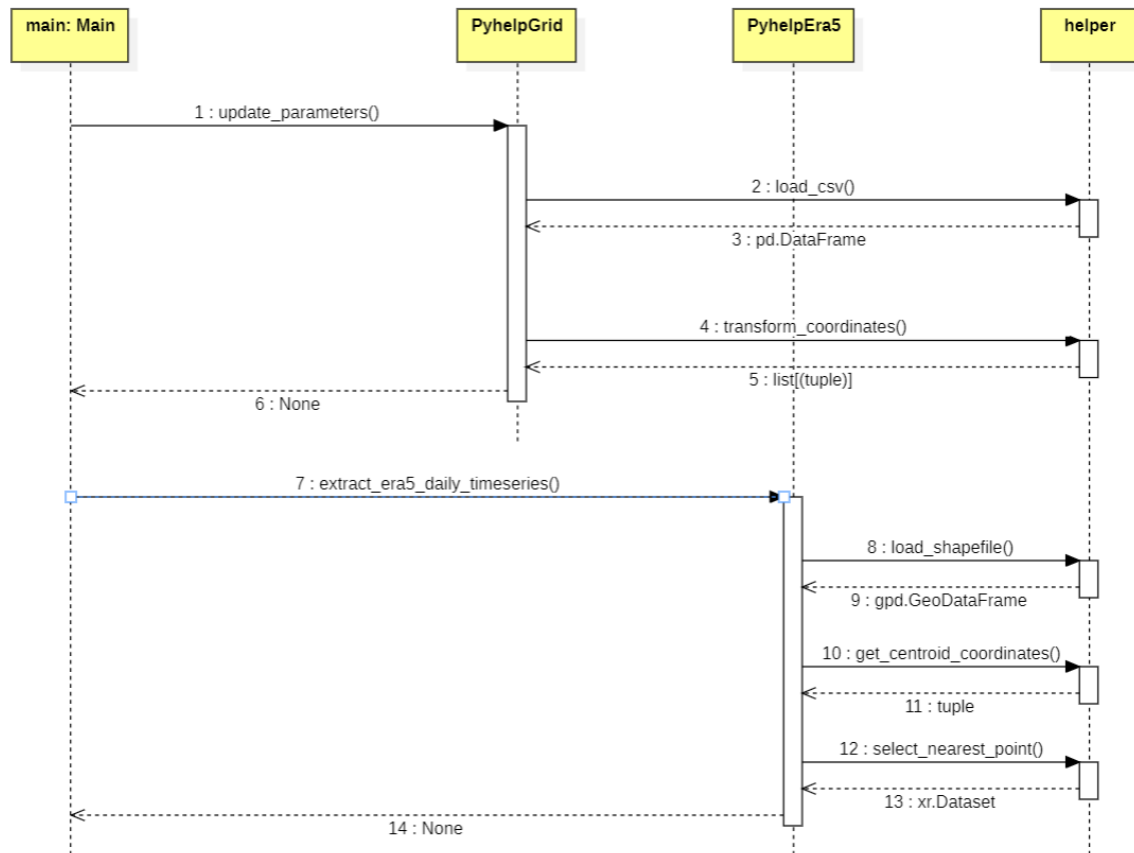
3.4 ERA5 (netCDF files)

The `era5_folder_path` folder contains three subfolders: radiation, precipitation and temperature.

Each is further divided by year (2020, 2021, etc.), containing the hourly NetCDF files. Make sure that each NetCDF has a `valid_time` dimension of size $24 \times N$, representing hourly time steps.

The script will open all .nc files for a given year, concatenate them, select the centroid (longitude, latitude) of the shapefile, perform a reshape of the timeserie to reduce the schedule to a daily average and rotate the structure into a dataframe, indexed by date and columns of latitude, longitude. As a finale step, the code will then convert units if necessary (Kelvin $\rightarrow$ °C, m $\rightarrow$ mm, J/m² $\rightarrow$ MJ/m²).

4) How the main script works



4.1 Loading and updating the grid

In main.py:

```
pyhelp_grid = PyhelpGrid(csv_file_path, dem_file_path)
pyhelp_grid.update_parameters(wind=99, growth_start=100)
```

Those lines do the following internal steps :

- 1) `PyhelpGrid.__init__`:
Calls `_load_data` which reads `input_grid_base.csv` (via `helper.load_csv`) and stores this `DataFrame` in `self._data`.
- 2) `Update_parameters(wind=99, growth_start=100)`
Set `wind` column to 99, `growth_start` to 100 (here those parameters were chosen as an exemple).and builds a list of coordinates (longitude, latitude) for each DEM pixel (via `transform_coordinates`). The code then injects these longitude and latitude coordinates into the `DataFrame` and saves the updated CSV.

As a result of these steps, the `input_grid_base.csv` is now loaded with latitude and longitude coordinates and possibly new parameters.

4.2 ERA5 data extraction and aggregation

In main.py:

```
pyhelp_era5 = PyhelpEra5(era5_folder_path, shp_file_path)
pyhelp_era5.extract_era5_daily_timeseries()
```

which performs the following steps:

- 1) `PyhelpEra5.__init__`:
Stores `era5_folder_path` and `shapefile_path`.
- 2) `extract_era5_daily_timeseries()`
For each variable (radiation, precipitation, temperature), fetch the corresponding folder (`era5_folder_path/radiation`, etc.) and browse the annual sub-folders (2020, 2021, etc.). For each year, the function will open all NetCDFs.
Through internal private functions, the code will then load the shapefile, calculates the centroid and selects the corresponding point in the dataset. The dataset will be aggregated into daily averages and then convert the units according to variable (e.g. Kelvin $\rightarrow$ $^{\circ}\text{C}$, m $\rightarrow$ mm, etc.). Finally, a final CSV is saved (e.g. `precip_input_data.csv`).

4.3 Final CSV file format

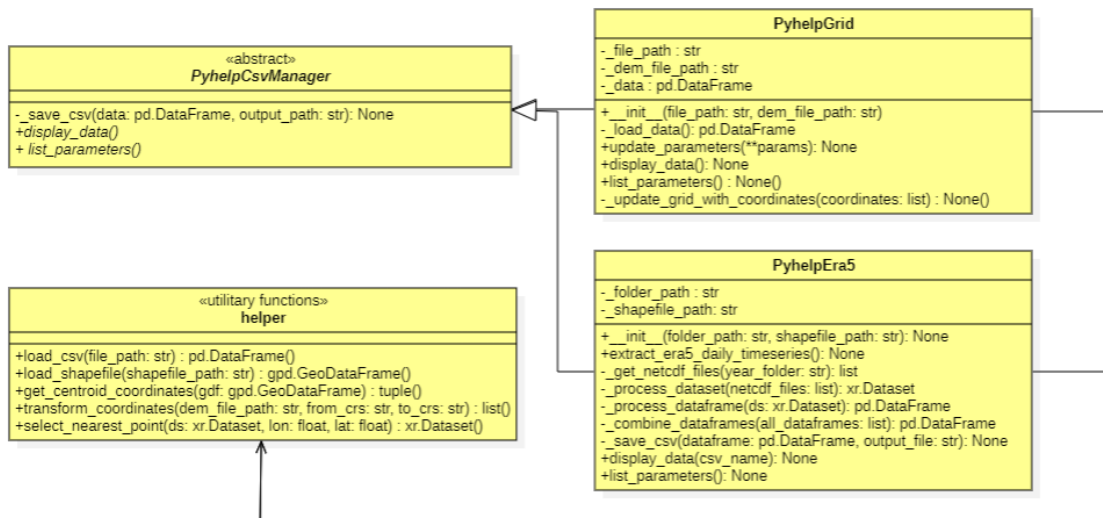
The final CSV file for the input grid will look as follows:

	cid	lat_dd	lon_dd	wind	...	BT	Urban	Culture	UtilSolCod
0	99	74.152395	-67.141984	99	...	400.0	23.0	74.152395	-67.141984
1	99	74.152869	-67.141986	99	...	400.0	23.0	74.152869	-67.141986
2	99	74.153342	-67.141988	99	...	400.0	23.0	74.153342	-67.141988
3	99	74.153815	-67.141990	99	...	400.0	23.0	74.153815	-67.141990
4	99	74.154289	-67.141992	99	...	400.0	23.0	74.154289	-67.141992
...	...	...	...	...	...	...	...	...	...
975	99	74.166580	-67.188863	99	...	400.0	23.0	74.166580	-67.188863
976	99	74.167054	-67.188866	99	...	400.0	23.0	74.167054	-67.188866
977	99	74.167527	-67.188869	99	...	400.0	23.0	74.167527	-67.188869
978	99	74.168001	-67.188872	99	...	400.0	23.0	74.168001	-67.188872
979	99	74.168474	-67.188875	99	...	400.0	23.0	74.168474	-67.188875

The final CSV file for the weather data will look as follows (example from precipitation data):

	Latitude (dd)	46.25
0	Longitude (dd)	10.000000
1	01/01/2000	0.099222
2	02/01/2000	0.075526
3	03/01/2000	0.003233
4	04/01/2000	0.000999
...	...	...
4014	27/12/2010	0.095697
4015	28/12/2010	0.301148
4016	29/12/2010	0.089921
4017	30/12/2010	0.009729
4018	31/12/2010	0.002116

5) Description of modules



5.1 helper.py

This module contains the following functions:

- `load_csv`, `load_shapefile`: load a CSV and shapefile respectively.
- `get_centroid_coordinates`: retrieves the centroid of the first geometry in a GeoDataFrame.
- `transform_coordinates`: reads a DEM raster and produces a list of latitude and longitude for each pixel, transforming from a source EPSG to a target EPSG. For example 2056 to 4326.
- `select_nearest_point`: in a Dataset Xarray, selects the nearest longitude, latitude point.
- `convert_units`: handles unit conversion for temperature, precipitation and radiation.

5.2 pyhelp_csv_manager.py

This module defines an abstract class which imposes the following methods:

- `_save_csv(data, output_path)`: used to save a DataFrame as CSV file.
- `display_data()`: abstract method (each subclass will implement it).
- `list_parameters()`: same as `display_data()`

Any class inheriting from `PyhelpCsvManager` will provide a display method and a list of parameters.

5.3 pyhelp_grid.py

The class from this module is inherited from `PyhelpCsvManager`. Its purpose is to manage the PyHelp grid CSV file. Here are its main methods:

- `_load_data()`: loads the data within the CSV file via `helper.load_csv`.
- `update_parameters(**params)`: this method updates the dataframe columns (wind, hum1, etc.), then restarts `_update_grid_with_coordinates()`.
- `_update_grid_with_coordinates(coordinates)`: from a list of latitude and longitude, the function fills "lon" and "lat" columns, then calls `_save_csv` to rewrite the file.

5.4 pyhelp_era5.py

Also inherited from `PyhelpCsvManager`, the purpose of this class is to extract and convert hourly ERA5 data into independent daily CSV files. Its main methods are:

- `extract_era5_daily_timeseries()`: the “pilot” method, which traverses all variable/year folders, creates a DataFrame, concatenates it and writes a final CSV for each weather parameter.
- `_get_nearest_point(ds)`: uses the shapefile to locate the area of interest.
- `_process_dataset(netcdf_files)`: opens NetCDF, reshape the hourly data into daily data.
- `_process_dataframe(ds)`: switches from Xarray dataset to rotated dataframe.
- `_combine_dataframes(all_dataframes)`: concatenates different annual dataframes and format the index in dd/mm/yyyy.
- `_save_csv(dataframe, output_file)`: writes the final CSV file and save the result

5.5 main.py

This is the main script that creates a PyhelpGrid object to load and update the grid. It creates a PyhelpEra5 object to extract NetCDF data and launches the `extract_era5_daily_timeseries()` method.

6) Outputs

6.1 Generated CSV Files (ERA5 Outputs)

After the run is completed, the following three new CSV files appear in the output file:

- 1) `solrad_input_data.csv` (radiation)
- 2) `precip_input_data.csv` (precipitation)
- 3) `airtemp_input_data.csv` (temperature)

those files can then be used as an input to run the PyHelp model.

6.2 Grid CSV File Updates

The `input_grid_base1.csv` is overwritten with new columns:

- lat, lon (transformed from the DEM).
- Any parameter overrides you set (wind, hum1, growth_start, etc.)